

Line-by-Line Mathematical Guide for HAETAE_Keccak1600.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Keccak state and low-level hash support.

How To Read This Script

Read this file as finite-state deterministic support for the modeled hash layer.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore IntDiv List.</code>	Imports AllCore IntDiv List. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
3	<code>theory HAETAE_Keccak1600.</code>	Begins the namespace HAETAE_Keccak1600, grouping the declarations as one checked theory.
4	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
5	<code>type keccak_byte = int.</code>	Declares carrier set keccak_byte. EasyCrypt will type-check all later functions and games over this mathematical domain.
6	<code>type keccak_lane = bool list.</code>	Declares carrier set keccak_lane. EasyCrypt will type-check all later functions and games over this mathematical domain.
7	<code>type keccak_state = keccak_lane list.</code>	Declares carrier set keccak_state. EasyCrypt will type-check all later functions and games over this mathematical domain.
8	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
9	<code>op keccak_lane_bits : int = 64.</code>	Defines integer-valued parameter keccak_lane_bits, used for dimensions, budgets, support sizes, or bounds.
10	<code>op keccak_state_lanes : int = 25.</code>	Defines integer-valued parameter keccak_state_lanes, used for dimensions, budgets, support sizes, or bounds.
11	<code>op keccak_state_bytes : int = 200.</code>	Defines integer-valued parameter keccak_state_bytes, used for dimensions, budgets, support sizes, or bounds.
12	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
13	<code>op keccak_byte_norm (b : keccak_byte) : keccak_byte = b %% 256.</code>	Defines operation keccak_byte_norm, a total mathematical function or abbreviation used by later declarations.
14	<code>op keccak_byte_bit (b : keccak_byte) (i : int) : bool =</code>	Defines predicate keccak_byte_bit, an event or well-formedness condition used in later implications and game proofs.

Line	EasyCrypt source	Mathematical reading
15	((keccak_byte_norm b) %/ (2 ^ i)) %% 2 <> 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
16	op keccak_word_norm (w : int) : int = w %% (2 ^ keccak_lane_bits).	Defines integer-valued parameter keccak_word_norm, used for dimensions, budgets, support sizes, or bounds.
17	op keccak_word_bit (w : int) (i : int) : bool =	Defines predicate keccak_word_bit, an event or well-formedness condition used in later implications and game proofs.
18	((keccak_word_norm w) %/ (2 ^ i)) %% 2 <> 0.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
19	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
20	op keccak_lane_zero : keccak_lane = nseq keccak_lane_bits false.	Defines operation keccak_lane_zero, a total mathematical function or abbreviation used by later declarations.
21	op keccak_lane_bit (lane : keccak_lane) (i : int) : bool =	Defines predicate keccak_lane_bit, an event or well-formedness condition used in later implications and game proofs.
22	nth false lane i.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
23	op keccak_lane_wf (lane : keccak_lane) : bool =	Defines predicate keccak_lane_wf, an event or well-formedness condition used in later implications and game proofs.
24	size lane = keccak_lane_bits.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
25	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
26	op keccak_lane_xor (a b : keccak_lane) : keccak_lane =	Defines operation keccak_lane_xor, a total mathematical function or abbreviation used by later declarations.
27	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
28	(fun i => keccak_lane_bit a i <> keccak_lane_bit b i)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
29	keccak_lane_bits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	op keccak_lane_and (a b : keccak_lane) : keccak_lane =	Defines operation keccak_lane_and, a total mathematical function or abbreviation used by later declarations.
31	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
32	(fun i => keccak_lane_bit a i /\ keccak_lane_bit b i)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
33	keccak_lane_bits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
34	op keccak_lane_not (a : keccak_lane) : keccak_lane =	Defines operation keccak_lane_not, a total mathematical function or abbreviation used by later declarations.
35	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
36	(fun i => if keccak_lane_bit a i then false else true)	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
37	keccak_lane_bits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
38	op keccak_lane_rotl (a : keccak_lane) (offset : int) : keccak_lane =	Defines integer-valued parameter keccak_lane_rotl, used for dimensions, budgets, support sizes, or bounds.
39	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
40	(fun i => keccak_lane_bit a ((i - offset) %% keccak_lane_bits))	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
41	keccak_lane_bits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
42	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
43	op keccak_lane_of_int (x : int) : keccak_lane =	Defines integer-valued parameter keccak_lane_of_int, used for dimensions, budgets, support sizes, or bounds.
44	mkseq (fun i => keccak_word_bit x i) keccak_lane_bits.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
45	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
46	op keccak_lane_to_byte (lane : keccak_lane) (byte_index : int) : keccak_byte =	Defines integer-valued parameter keccak_lane_to_byte, used for dimensions, budgets, support sizes, or bounds.
47	sumz	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
48	<code>(map</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
49	<code>(fun j =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
50	<code>if keccak_lane_bit lane (8 * byte_index + j)</code>	Splits proof obligations according to a conditional branch in the program or formula.
51	<code>then 2 ^ j</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
52	<code>else 0)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	<code>(range 0 8)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
55	<code>op keccak_bytes_to_lane (bs : keccak_byte list) (lane_index : int) :</code>	Defines integer-valued parameter <code>keccak_bytes_to_lane</code> , used for dimensions, budgets, support sizes, or bounds.
56	<code>keccak_lane =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
57	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
58	<code>(fun bit =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
59	<code>keccak_byte_bit</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
60	<code>(nth 0 bs (8 * lane_index + bit % 8))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
61	<code>(bit % 8))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
62	<code>keccak_lane_bits.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
63	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
64	<code>op keccak_lanes_of_bytes (bs : keccak_byte list) : keccak_state =</code>	Defines operation <code>keccak_lanes_of_bytes</code> , a total mathematical function or abbreviation used by later declarations.
65	<code>mkseq (fun lane => keccak_bytes_to_lane bs lane) keccak_state_lanes.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
66	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
67	<code>op keccak_bytes_of_lanes (st : keccak_state) : keccak_byte list =</code>	Defines operation <code>keccak_bytes_of_lanes</code> , a total mathematical function or abbreviation used by later declarations.
68	<code>mkseq</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	<code>(fun i =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
70	<code>keccak_lane_to_byte</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
71	<code>(nth keccak_lane_zero st (i % 8))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
72	<code>(i % 8))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
73	<code>keccak_state_bytes.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
74	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
75	<code>op keccak_lane_index (x y : int) : int =</code>	Defines integer-valued parameter <code>keccak_lane_index</code> , used for dimensions, budgets, support sizes, or bounds.
76	<code>(x % 5) + 5 * (y % 5).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
77	<code>op keccak_state_lane (st : keccak_state) (x y : int) : keccak_lane =</code>	Defines integer-valued parameter <code>keccak_state_lane</code> , used for dimensions, budgets, support sizes, or bounds.
78	<code>nth keccak_lane_zero st (keccak_lane_index x y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
79	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
80	<code>op keccak_rho_offsets : int list =</code>	Defines integer-valued parameter <code>keccak_rho_offsets</code> , used for dimensions, budgets, support sizes, or bounds.

Line	EasyCrypt source	Mathematical reading
81	0 :: 1 :: 62 :: 28 :: 27 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
82	36 :: 44 :: 6 :: 55 :: 20 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
83	3 :: 10 :: 43 :: 25 :: 39 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
84	41 :: 45 :: 15 :: 21 :: 8 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
85	18 :: 2 :: 61 :: 56 :: 14 :: [].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
86	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
87	op keccak_round_constants : int list =	Defines integer-valued parameter keccak_round_constants, used for dimensions, budgets, support sizes, or bounds.
88	1 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
89	32898 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
90	9223372036854808714 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
91	9223372039002292224 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
92	32907 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
93	2147483649 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
94	9223372039002292353 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
95	9223372036854808585 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
96	138 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
97	136 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
98	2147516425 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
99	2147483658 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
100	2147516555 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
101	9223372036854775947 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
102	9223372036854808713 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
103	9223372036854808579 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
104	9223372036854808578 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
105	9223372036854775936 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
106	32778 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
107	9223372039002259466 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
108	9223372039002292353 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
109	9223372036854808704 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
110	2147483649 ::	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
111	9223372039002292232 :: [].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
112	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
113	op keccak_theta_c (st : keccak_state) (x : int) : keccak_lane =	Defines integer-valued parameter keccak_theta_c, used for dimensions, budgets, support sizes, or bounds.

Line	EasyCrypt source	Mathematical reading
114	keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
115	(keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
116	(keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
117	(keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
118	(keccak_state_lane st x 0)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
119	(keccak_state_lane st x 1))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
120	(keccak_state_lane st x 2))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
121	(keccak_state_lane st x 3))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
122	(keccak_state_lane st x 4).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
123	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
124	op keccak_theta_d (st : keccak_state) (x : int) : keccak_lane =	Defines integer-valued parameter keccak_theta_d, used for dimensions, budgets, support sizes, or bounds.
125	keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
126	(keccak_theta_c st (x + 4))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
127	(keccak_lane_rotl (keccak_theta_c st (x + 1)) 1).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
128	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
129	op keccak_theta (st : keccak_state) : keccak_state =	Defines operation keccak_theta, a total mathematical function or abbreviation used by later declarations.
130	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
131	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
132	let x = i %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
133	let y = i %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
134	keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
135	(keccak_state_lane st x y)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
136	(keccak_theta_d st x))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
137	keccak_state_lanes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
138	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
139	op keccak_rho_pi (st : keccak_state) : keccak_state =	Defines operation keccak_rho_pi, a total mathematical function or abbreviation used by later declarations.
140	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
141	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
142	let x = i %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
143	let y = i %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
144	let sx = (x + 3 * y) %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
145	let sy = x in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
146	keccak_lane_rotl	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
147	(keccak_state_lane st sx sy)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
148	(nth 0 keccak_rho_offsets (keccak_lane_index sx sy)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
149	keccak_state_lanes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
150	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
151	op keccak_chi (st : keccak_state) : keccak_state =	Defines operation keccak_chi, a total mathematical function or abbreviation used by later declarations.
152	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
153	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
154	let x = i %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
155	let y = i %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
156	keccak_lane_xor	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
157	(keccak_state_lane st x y)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
158	(keccak_lane_and	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
159	(keccak_lane_not (keccak_state_lane st (x + 1) y))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
160	(keccak_state_lane st (x + 2) y)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
161	keccak_state_lanes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
162	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
163	op keccak_iota (st : keccak_state) (rc : keccak_lane) : keccak_state =	Defines operation keccak_iota, a total mathematical function or abbreviation used by later declarations.
164	mkseq	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
165	(fun i =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
166	if i = 0	Splits proof obligations according to a conditional branch in the program or formula.
167	then keccak_lane_xor (nth keccak_lane_zero st 0) rc	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
168	else nth keccak_lane_zero st i)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
169	keccak_state_lanes.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
170	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
171	op keccak_round (st : keccak_state) (rc : keccak_lane) : keccak_state =	Defines operation keccak_round, a total mathematical function or abbreviation used by later declarations.
172	keccak_iota (keccak_chi (keccak_rho_pi (keccak_theta st))) rc.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
173	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
174	op keccak_f1600_lanes (st : keccak_state) : keccak_state =	Defines operation keccak_f1600_lanes, a total mathematical function or abbreviation used by later declarations.
175	foldl	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
176	(fun acc rc => keccak_round acc (keccak_lane_of_int rc))	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
177	st	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
178	keccak_round_constants.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
179	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
180	op keccak_f1600_bytes (st : keccak_byte list) : keccak_byte list =	Defines operation keccak_f1600_bytes, a total mathematical function or abbreviation used by later declarations.
181	keccak_bytes_of_lanes (keccak_f1600_lanes (keccak_lanes_of_bytes st)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
182	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
183	lemma keccak_lane_bitsE :	States checked lemma keccak_lane_bitsE. The following proof script must derive this proposition from prior definitions and lemmas.
184	keccak_lane_bits = 64.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
185	proof. by rewrite /keccak_lane_bits. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
186	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
187	lemma keccak_state_lanesE :	States checked lemma keccak_state_lanesE. The following proof script must derive this proposition from prior definitions and lemmas.
188	keccak_state_lanes = 25.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
189	proof. by rewrite /keccak_state_lanes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
190	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
191	lemma keccak_state_bytesE :	States checked lemma keccak_state_bytesE. The following proof script must derive this proposition from prior definitions and lemmas.
192	keccak_state_bytes = 200.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
193	proof. by rewrite /keccak_state_bytes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
194	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
195	lemma keccak_lane_zero_wf :	States checked lemma keccak_lane_zero_wf. The following proof script must derive this proposition from prior definitions and lemmas.
196	keccak_lane_wf keccak_lane_zero.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
197	proof. by rewrite /keccak_lane_wf /keccak_lane_zero size_nseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
198	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
199	lemma keccak_lane_xor_wf a b :	States checked lemma keccak_lane_xor_wf. The following proof script must derive this proposition from prior definitions and lemmas.
200	keccak_lane_wf (keccak_lane_xor a b).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
201	proof. by rewrite /keccak_lane_wf /keccak_lane_xor size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
202	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
203	lemma keccak_lane_and_wf a b :	States checked lemma keccak_lane_and_wf. The following proof script must derive this proposition from prior definitions and lemmas.
204	keccak_lane_wf (keccak_lane_and a b).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
205	proof. by rewrite /keccak_lane_wf /keccak_lane_and size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
206	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
207	lemma keccak_lane_not_wf a :	States checked lemma keccak_lane_not_wf. The following proof script must derive this proposition from prior definitions and lemmas.
208	keccak_lane_wf (keccak_lane_not a).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
209	proof. by rewrite /keccak_lane_wf /keccak_lane_not size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
210	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
211	lemma keccak_lane_rotl_wf a offset :	States checked lemma keccak_lane_rotl_wf. The following proof script must derive this proposition from prior definitions and lemmas.
212	keccak_lane_wf (keccak_lane_rotl a offset).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
213	proof. by rewrite /keccak_lane_wf /keccak_lane_rotl size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
214	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
215	lemma keccak_lane_of_int_wf x :	States checked lemma keccak_lane_of_int_wf. The following proof script must derive this proposition from prior definitions and lemmas.
216	keccak_lane_wf (keccak_lane_of_int x).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
217	proof. by rewrite /keccak_lane_wf /keccak_lane_of_int size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
218	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
219	lemma keccak_lane_mod_range x :	States checked lemma keccak_lane_mod_range. The following proof script must derive this proposition from prior definitions and lemmas.
220	0 <= x %% keccak_lane_bits < keccak_lane_bits.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
221	proof. by rewrite /keccak_lane_bits; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
222	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
223	lemma keccak_word_modulus_pos :	States checked lemma keccak_word_modulus_pos. The following proof script must derive this proposition from prior definitions and lemmas.
224	0 < 2 ^ keccak_lane_bits.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
225	proof. by rewrite /keccak_lane_bits; smt(). qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
226	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
227	lemma keccak_word_norm_small w :	States checked lemma keccak_word_norm_small. The following proof script must derive this proposition from prior definitions and lemmas.
228	0 <= w < 2 ^ keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
229	keccak_word_norm w = w.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
230	proof.	Begins the proof script for the preceding proposition.
231	move=> w_range.	Introduces universally quantified variables or hypotheses into the proof context.
232	by rewrite /keccak_word_norm pmod_small.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
233	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
234	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
235	lemma keccak_pow2_0E :	States checked lemma keccak_pow2_0E. The following proof script must derive this proposition from prior definitions and lemmas.
236	2 ^ 0 = 1.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
237	proof. by rewrite expr0. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
238	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
239	lemma keccak_pow2_1E :	States checked lemma keccak_pow2_1E. The following proof script must derive this proposition from prior definitions and lemmas.
240	2 ^ 1 = 2.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
241	proof. by rewrite expr1. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
242	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
243	lemma keccak_pow2_2E :	States checked lemma keccak_pow2_2E. The following proof script must derive this proposition from prior definitions and lemmas.
244	2 ^ 2 = 4.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
245	proof. by rewrite expr2. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
246	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
247	lemma keccak_pow2_3E :	States checked lemma keccak_pow2_3E. The following proof script must derive this proposition from prior definitions and lemmas.
248	2 ^ 3 = 8.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
249	proof.	Begins the proof script for the preceding proposition.
250	rewrite (_ : 3 = 2 + 1) 1:/# exprS //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
251	by rewrite keccak_pow2_2E.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
252	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
253	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
254	lemma keccak_pow2_4E :	States checked lemma keccak_pow2_4E. The following proof script must derive this proposition from prior definitions and lemmas.
255	2 ^ 4 = 16.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
256	proof.	Begins the proof script for the preceding proposition.
257	rewrite (_ : 4 = 3 + 1) 1:/# exprS //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
258	by rewrite keccak_pow2_3E.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
259	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
260	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
261	lemma keccak_pow2_5E :	States checked lemma keccak_pow2_5E. The following proof script must derive this proposition from prior definitions and lemmas.
262	2 ^ 5 = 32.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
263	proof.	Begins the proof script for the preceding proposition.
264	rewrite (_ : 5 = 4 + 1) 1:/# exprS //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
265	by rewrite keccak_pow2_4E.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
266	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
267	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
268	lemma keccak_pow2_6E :	States checked lemma keccak_pow2_6E. The following proof script must derive this proposition from prior definitions and lemmas.
269	2 ^ 6 = 64.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
270	proof.	Begins the proof script for the preceding proposition.
271	rewrite (_ : 6 = 5 + 1) 1:/# exprS //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
272	by rewrite keccak_pow2_5E.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
273	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
274	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
275	lemma keccak_pow2_7E :	States checked lemma keccak_pow2_7E. The following proof script must derive this proposition from prior definitions and lemmas.
276	2 ^ 7 = 128.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
277	proof.	Begins the proof script for the preceding proposition.
278	rewrite (_ : 7 = 6 + 1) 1:/# exprS //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
279	by rewrite keccak_pow2_6E.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
280	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
281	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
282	lemma keccak_bytes_to_lane_wf bs lane_index :	States checked lemma keccak_bytes_to_lane_wf. The following proof script must derive this proposition from prior definitions and lemmas.
283	keccak_lane_wf (keccak_bytes_to_lane bs lane_index).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
284	proof. by rewrite /keccak_lane_wf /keccak_bytes_to_lane size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
285	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
286	lemma keccak_rho_offsets_size :	States checked lemma keccak_rho_offsets_size. The following proof script must derive this proposition from prior definitions and lemmas.
287	size keccak_rho_offsets = keccak_state_lanes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
288	proof. by rewrite /keccak_rho_offsets /keccak_state_lanes. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
289	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
290	lemma keccak_round_constants_size :	States checked lemma keccak_round_constants_size. The following proof script must derive this proposition from prior definitions and lemmas.
291	size keccak_round_constants = 24.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
292	proof. by rewrite /keccak_round_constants. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
293	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
294	lemma keccak_lanes_of_bytes_size bs :	States checked lemma keccak_lanes_of_bytes_size. The following proof script must derive this proposition from prior definitions and lemmas.
295	size (keccak_lanes_of_bytes bs) = keccak_state_lanes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
296	proof. by rewrite /keccak_lanes_of_bytes size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
297	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
298	lemma keccak_lanes_of_bytes_lane_wf bs lane :	States checked lemma keccak_lanes_of_bytes_lane_wf. The following proof script must derive this proposition from prior definitions and lemmas.
299	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
300	keccak_lane_wf	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
301	(nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
302	proof.	Begins the proof script for the preceding proposition.
303	move=> lane_range.	Introduces universally quantified variables or hypotheses into the proof context.
304	rewrite /keccak_lanes_of_bytes nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
305	by apply keccak_bytes_to_lane_wf.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
306	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
307	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
308	lemma keccak_lanes_of_bytes_bitE bs lane bit :	States checked lemma keccak_lanes_of_bytes_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
309	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
310	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
311	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
312	(nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
313	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
314	keccak_byte_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
315	(nth 0 bs (8 * lane + bit % 8))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
316	(bit % 8).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
317	proof.	Begins the proof script for the preceding proposition.
318	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
319	rewrite /keccak_lane_bit /keccak_lanes_of_bytes nth_mkseq;	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
320	<code>first by smt().</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
321	<code>by rewrite /keccak_bytes_to_lane nth_mkseq.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
322	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
323	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
324	<code>lemma keccak_lane_xor_bitE a b bit :</code>	States checked lemma <code>keccak_lane_xor_bitE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
325	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
326	<code>keccak_lane_bit (keccak_lane_xor a b) bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
327	<code>(keccak_lane_bit a bit <> keccak_lane_bit b bit).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
328	<code>proof.</code>	Begins the proof script for the preceding proposition.
329	<code>move=> bit_range.</code>	Introduces universally quantified variables or hypotheses into the proof context.
330	<code>rewrite /keccak_lane_bit /keccak_lane_xor nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
331	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
332	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
333	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
334	<code>lemma keccak_lane_and_bitE a b bit :</code>	States checked lemma <code>keccak_lane_and_bitE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
335	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
336	<code>keccak_lane_bit (keccak_lane_and a b) bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
337	<code>(keccak_lane_bit a bit /\ keccak_lane_bit b bit).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
338	<code>proof.</code>	Begins the proof script for the preceding proposition.
339	<code>move=> bit_range.</code>	Introduces universally quantified variables or hypotheses into the proof context.
340	<code>rewrite /keccak_lane_bit /keccak_lane_and nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
341	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
342	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
343	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
344	<code>lemma keccak_lane_not_bitE a bit :</code>	States checked lemma <code>keccak_lane_not_bitE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
345	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
346	<code>keccak_lane_bit (keccak_lane_not a) bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
347	<code>(if keccak_lane_bit a bit then false else true).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
348	<code>proof.</code>	Begins the proof script for the preceding proposition.
349	<code>move=> bit_range.</code>	Introduces universally quantified variables or hypotheses into the proof context.
350	<code>rewrite /keccak_lane_bit /keccak_lane_not nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
351	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
352	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
353	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
354	<code>lemma keccak_lane_rotl_bitE a offset bit :</code>	States checked lemma <code>keccak_lane_rotl_bitE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
355	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
356	<code>keccak_lane_bit (keccak_lane_rotl a offset) bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
357	keccak_lane_bit a ((bit - offset) %% keccak_lane_bits).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
358	proof.	Begins the proof script for the preceding proposition.
359	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
360	rewrite /keccak_lane_bit /keccak_lane_rotl nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
361	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
362	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
363	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
364	lemma keccak_lane_of_int_bitE x bit :	States checked lemma keccak_lane_of_int_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
365	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
366	keccak_lane_bit (keccak_lane_of_int x) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
367	keccak_word_bit x bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
368	proof.	Begins the proof script for the preceding proposition.
369	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
370	rewrite /keccak_lane_bit /keccak_lane_of_int nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
371	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
372	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
373	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
374	lemma keccak_state_lane_bitE st x y bit :	States checked lemma keccak_state_lane_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
375	keccak_lane_bit (keccak_state_lane st x y) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
376	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
377	(nth keccak_lane_zero st (keccak_lane_index x y))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
378	bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
379	proof. by rewrite /keccak_state_lane. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
380	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
381	op keccak_theta_c_bit_index_value	Defines operation keccak_theta_c_bit_index_value, a total mathematical function or abbreviation used by later declarations.
382	(st : keccak_state) (x bit : int) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
383	(((((keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
384	(nth keccak_lane_zero st (keccak_lane_index x 0)) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
385	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
386	(nth keccak_lane_zero st (keccak_lane_index x 1)) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
387	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
388	(nth keccak_lane_zero st (keccak_lane_index x 2)) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
389	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
390	(nth keccak_lane_zero st (keccak_lane_index x 3)) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
391	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
392	(nth keccak_lane_zero st (keccak_lane_index x 4)) bit).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
393	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
394	op keccak_theta_d_bit_index_value	Defines operation keccak_theta_d_bit_index_value, a total mathematical function or abbreviation used by later declarations.
395	(st : keccak_state) (x bit : int) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
396	keccak_theta_c_bit_index_value st (x + 4) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
397	keccak_theta_c_bit_index_value	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
398	st	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
399	(x + 1)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
400	((bit - 1) %% keccak_lane_bits).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
401	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
402	lemma keccak_theta_c_bitE st x bit :	States checked lemma keccak_theta_c_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
403	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
404	keccak_lane_bit (keccak_theta_c st x) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
405	((((keccak_lane_bit (keccak_state_lane st x 0) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
406	keccak_lane_bit (keccak_state_lane st x 1) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
407	keccak_lane_bit (keccak_state_lane st x 2) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
408	keccak_lane_bit (keccak_state_lane st x 3) bit) <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
409	keccak_lane_bit (keccak_state_lane st x 4) bit).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
410	proof.	Begins the proof script for the preceding proposition.
411	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
412	by rewrite /keccak_theta_c !keccak_lane_xor_bitE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
413	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
414	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
415	lemma keccak_theta_c_bit_indexE st x bit :	States checked lemma keccak_theta_c_bit_indexE. The following proof script must derive this proposition from prior definitions and lemmas.
416	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
417	keccak_lane_bit (keccak_theta_c st x) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
418	keccak_theta_c_bit_index_value st x bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
419	proof.	Begins the proof script for the preceding proposition.
420	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
421	by rewrite /keccak_theta_c_bit_index_value	Discharges the current goal in one step using the cited simplification, lemma, or automation.
422	keccak_theta_c_bitE // !keccak_state_lane_bitE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
423	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
424	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
425	lemma keccak_theta_d_bitE st x bit :	States checked lemma keccak_theta_d_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
426	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
427	keccak_lane_bit (keccak_theta_d st x) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
428	(keccak_lane_bit (keccak_theta_c st (x + 4)) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
429	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
430	(keccak_theta_c st (x + 1))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
431	((bit - 1) %% keccak_lane_bits)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
432	proof.	Begins the proof script for the preceding proposition.
433	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
434	rewrite /keccak_theta_d.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
435	rewrite keccak_lane_xor_bitE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
436	by rewrite keccak_lane_rotl_bitE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
437	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
438	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
439	lemma keccak_theta_d_bit_indexE st x bit :	States checked lemma keccak_theta_d_bit_indexE. The following proof script must derive this proposition from prior definitions and lemmas.
440	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
441	keccak_lane_bit (keccak_theta_d st x) bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
442	keccak_theta_d_bit_index_value st x bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
443	proof.	Begins the proof script for the preceding proposition.
444	move=> bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
445	rewrite keccak_theta_d_bitE //.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
446	rewrite /keccak_theta_d_bit_index_value.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
447	rewrite keccak_theta_c_bit_indexE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
448	rewrite keccak_theta_c_bit_indexE; first by rewrite /keccak_lane_bits; smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
449	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
450	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
451	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
452	lemma keccak_theta_bitE st lane bit :	States checked lemma keccak_theta_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
453	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
454	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
455	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
456	(nth keccak_lane_zero (keccak_theta st) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
457	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
458	(let x = lane %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
459	let y = lane %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
460	keccak_lane_bit (keccak_state_lane st x y) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
461	keccak_lane_bit (keccak_theta_d st x) bit).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
462	<code>proof.</code>	Begins the proof script for the preceding proposition.
463	<code>move=> lane_range bit_range.</code>	Introduces universally quantified variables or hypotheses into the proof context.
464	<code>rewrite /keccak_theta nth_mkseq; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
465	<code>rewrite /= keccak_lane_xor_bitE; first by smt().</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
466	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
467	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
468	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
469	<code>lemma keccak_theta_bit_indexE st lane bit :</code>	States checked lemma <code>keccak_theta_bit_indexE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
470	<code>0 <= lane < keccak_state_lanes =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
471	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
472	<code>keccak_lane_bit</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
473	<code>(nth keccak_lane_zero (keccak_theta st) lane)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
474	<code>bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
475	<code>(let x = lane %% 5 in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
476	<code>let y = lane %/ 5 in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
477	<code>keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
478	<code>keccak_lane_bit (keccak_theta_d st x) bit).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
479	<code>proof.</code>	Begins the proof script for the preceding proposition.
480	<code>move=> lane_range bit_range.</code>	Introduces universally quantified variables or hypotheses into the proof context.
481	<code>by rewrite keccak_theta_bitE // keccak_state_lane_bitE.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
482	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
483	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
484	<code>lemma keccak_rho_pi_bitE st lane bit :</code>	States checked lemma <code>keccak_rho_pi_bitE</code> . The following proof script must derive this proposition from prior definitions and lemmas.
485	<code>0 <= lane < keccak_state_lanes =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
486	<code>0 <= bit < keccak_lane_bits =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
487	<code>keccak_lane_bit</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
488	<code>(nth keccak_lane_zero (keccak_rho_pi st) lane)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
489	<code>bit =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
490	<code>(let x = lane %% 5 in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
491	<code>let y = lane %/ 5 in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
492	<code>let sx = (x + 3 * y) %% 5 in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
493	<code>let sy = x in</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
494	<code>keccak_lane_bit</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
495	<code>(keccak_state_lane st sx sy)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
496	<code>((bit - nth 0 keccak_rho_offsets (keccak_lane_index sx sy)) %%</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
497	keccak_lane_bits)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
498	proof.	Begins the proof script for the preceding proposition.
499	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
500	rewrite /keccak_rho_pi nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
501	rewrite /= keccak_lane_rotl_bitE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
502	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
503	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
504	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
505	lemma keccak_rho_pi_bit_indexE st lane bit :	States checked lemma keccak_rho_pi_bit_indexE. The following proof script must derive this proposition from prior definitions and lemmas.
506	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
507	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
508	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
509	(nth keccak_lane_zero (keccak_rho_pi st) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
510	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
511	(let x = lane %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
512	let y = lane %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
513	let sx = (x + 3 * y) %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
514	let sy = x in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
515	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
516	(nth keccak_lane_zero st (keccak_lane_index sx sy))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
517	((bit - nth 0 keccak_rho_offsets (keccak_lane_index sx sy)) %%	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
518	keccak_lane_bits)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
519	proof.	Begins the proof script for the preceding proposition.
520	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
521	by rewrite keccak_rho_pi_bitE // keccak_state_lane_bitE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
522	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
523	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
524	lemma keccak_chi_bitE st lane bit :	States checked lemma keccak_chi_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
525	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
526	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
527	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
528	(nth keccak_lane_zero (keccak_chi st) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
529	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
530	(let x = lane %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
531	let y = lane %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
532	keccak_lane_bit (keccak_state_lane st x y) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
533	((if keccak_lane_bit (keccak_state_lane st (x + 1) y) bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
534	then false else true) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
535	keccak_lane_bit (keccak_state_lane st (x + 2) y) bit)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
536	proof.	Begins the proof script for the preceding proposition.
537	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
538	rewrite /keccak_chi nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
539	rewrite /= keccak_lane_xor_bitE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
540	rewrite keccak_lane_and_bitE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
541	rewrite keccak_lane_not_bitE; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
542	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
543	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
544	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
545	lemma keccak_chi_bit_indexE st lane bit :	States checked lemma keccak_chi_bit_indexE. The following proof script must derive this proposition from prior definitions and lemmas.
546	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
547	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
548	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
549	(nth keccak_lane_zero (keccak_chi st) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
550	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
551	(let x = lane %% 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
552	let y = lane %/ 5 in	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
553	keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
554	((if keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
555	(nth keccak_lane_zero st (keccak_lane_index (x + 1) y))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
556	bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
557	then false else true) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
558	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
559	(nth keccak_lane_zero st (keccak_lane_index (x + 2) y))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
560	bit)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
561	proof.	Begins the proof script for the preceding proposition.
562	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
563	by rewrite keccak_chi_bitE // !keccak_state_lane_bitE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
564	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
565	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
566	lemma keccak_iota_bitE st rc lane bit :	States checked lemma keccak_iota_bitE. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
567	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
568	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
569	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
570	(nth keccak_lane_zero (keccak_iota st rc) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
571	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
572	if lane = 0	Splits proof obligations according to a conditional branch in the program or formula.
573	then keccak_lane_bit (nth keccak_lane_zero st 0) bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
574	keccak_lane_bit rc bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
575	else keccak_lane_bit (nth keccak_lane_zero st lane) bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
576	proof.	Begins the proof script for the preceding proposition.
577	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
578	rewrite /keccak_iota nth_mkseq; first by smt().	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
579	case (lane = 0) => lane0.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
580	+ rewrite lane0 /= keccak_lane_xor_bitE; first by smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
581	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
582	by rewrite lane0.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
583	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
584	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
585	lemma keccak_round_bitE st rc lane bit :	States checked lemma keccak_round_bitE. The following proof script must derive this proposition from prior definitions and lemmas.
586	0 <= lane < keccak_state_lanes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
587	0 <= bit < keccak_lane_bits =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
588	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
589	(nth keccak_lane_zero (keccak_round st rc) lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
590	bit =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
591	if lane = 0	Splits proof obligations according to a conditional branch in the program or formula.
592	then	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
593	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
594	(nth keccak_lane_zero	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
595	(keccak_chi (keccak_rho_pi (keccak_theta st)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
596	0)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
597	bit <>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
598	keccak_lane_bit rc bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
599	else	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
600	keccak_lane_bit	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
601	(nth keccak_lane_zero	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
602	(keccak_chi (keccak_rho_pi (keccak_theta st)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
603	lane)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
604	bit.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
605	proof.	Begins the proof script for the preceding proposition.
606	move=> lane_range bit_range.	Introduces universally quantified variables or hypotheses into the proof context.
607	rewrite /keccak_round.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
608	by rewrite keccak_iota_bitE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
609	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
610	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
611	lemma keccak_bytes_of_lanes_size st :	States checked lemma keccak_bytes_of_lanes_size. The following proof script must derive this proposition from prior definitions and lemmas.
612	size (keccak_bytes_of_lanes st) = keccak_state_bytes.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
613	proof. by rewrite /keccak_bytes_of_lanes size_mkseq. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
614	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
615	lemma keccak_bytes_of_lanes_byteE st i :	States checked lemma keccak_bytes_of_lanes_byteE. The following proof script must derive this proposition from prior definitions and lemmas.
616	0 <= i < keccak_state_bytes =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
617	nth 0 (keccak_bytes_of_lanes st) i =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
618	keccak_lane_to_byte	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
619	(nth keccak_lane_zero st (i % 8))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
620	(i % 8).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
621	proof.	Begins the proof script for the preceding proposition.
622	move=> i_range.	Introduces universally quantified variables or hypotheses into the proof context.
623	by rewrite /keccak_bytes_of_lanes nth_mkseq.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
624	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
625	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
626	lemma keccak_lane_to_byte_bitsE lane byte_index :	States checked lemma keccak_lane_to_byte_bitsE. The following proof script must derive this proposition from prior definitions and lemmas.
627	keccak_lane_to_byte lane byte_index =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
628	(if keccak_lane_bit lane (8 * byte_index) then 2 ^ 0 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
629	((if keccak_lane_bit lane (8 * byte_index + 1) then 2 ^ 1 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
630	((if keccak_lane_bit lane (8 * byte_index + 2) then 2 ^ 2 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
631	((if keccak_lane_bit lane (8 * byte_index + 3) then 2 ^ 3 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
632	((if keccak_lane_bit lane (8 * byte_index + 4) then 2 ^ 4 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
633	((if keccak_lane_bit lane (8 * byte_index + 5) then 2 ^ 5 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
634	((if keccak_lane_bit lane (8 * byte_index + 6) then 2 ^ 6 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
635	(if keccak_lane_bit lane (8 * byte_index + 7) then 2 ^ 7 else 0)))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
636	proof.	Begins the proof script for the preceding proposition.
637	rewrite /keccak_lane_to_byte /range /sumz /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
638	rewrite (@iotaS 0 7) 1:/# /=	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
639	(@iotaS 1 6) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
640	(@iotaS 2 5) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
641	(@iotaS 3 4) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
642	(@iotaS 4 3) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
643	(@iotaS 5 2) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
644	(@iotaS 6 1) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
645	(@iotaS 7 0) 1:/# /=	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
646	iota0 1:/# /.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
647	by [].	Discharges the current goal in one step using the cited simplification, lemma, or automation.
648	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
649	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
650	lemma keccak_lane_to_byte_from_bitsE	States checked lemma keccak_lane_to_byte_from_bitsE. The following proof script must derive this proposition from prior definitions and lemmas.
651	lane byte_index b0 b1 b2 b3 b4 b5 b6 b7 :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
652	keccak_lane_bit lane (8 * byte_index) = b0 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
653	keccak_lane_bit lane (8 * byte_index + 1) = b1 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
654	keccak_lane_bit lane (8 * byte_index + 2) = b2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
655	keccak_lane_bit lane (8 * byte_index + 3) = b3 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
656	keccak_lane_bit lane (8 * byte_index + 4) = b4 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
657	keccak_lane_bit lane (8 * byte_index + 5) = b5 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
658	keccak_lane_bit lane (8 * byte_index + 6) = b6 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
659	keccak_lane_bit lane (8 * byte_index + 7) = b7 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
660	keccak_lane_to_byte lane byte_index =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
661	(if b0 then 1 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
662	((if b1 then 2 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
663	((if b2 then 4 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
664	((if b3 then 8 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
665	((if b4 then 16 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
666	((if b5 then 32 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
667	((if b6 then 64 else 0) +	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
668	(if b7 then 128 else 0)))))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
669	proof.	Begins the proof script for the preceding proposition.
670	move=> h0 h1 h2 h3 h4 h5 h6 h7.	Introduces universally quantified variables or hypotheses into the proof context.

Line	EasyCrypt source	Mathematical reading
671	<code>rewrite keccak_lane_to_byte_bitsE h0 h1 h2 h3 h4 h5 h6 h7 /=. .</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
672	<code>by rewrite keccak_pow2_0E keccak_pow2_1E keccak_pow2_2E keccak_pow2_3E</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
673	<code>keccak_pow2_4E keccak_pow2_5E keccak_pow2_6E keccak_pow2_7E.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
674	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
675	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
676	<code>lemma keccak_lane_to_byte_70E lane byte_index :</code>	States checked lemma <code>keccak_lane_to_byte_70E</code> . The following proof script must derive this proposition from prior definitions and lemmas.
677	<code>keccak_lane_bit lane (8 * byte_index) = false =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
678	<code>keccak_lane_bit lane (8 * byte_index + 1) = true =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
679	<code>keccak_lane_bit lane (8 * byte_index + 2) = true =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
680	<code>keccak_lane_bit lane (8 * byte_index + 3) = false =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
681	<code>keccak_lane_bit lane (8 * byte_index + 4) = false =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
682	<code>keccak_lane_bit lane (8 * byte_index + 5) = false =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
683	<code>keccak_lane_bit lane (8 * byte_index + 6) = true =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
684	<code>keccak_lane_bit lane (8 * byte_index + 7) = false =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
685	<code>keccak_lane_to_byte lane byte_index = 70.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
686	<code>proof.</code>	Begins the proof script for the preceding proposition.
687	<code>move=> b0 b1 b2 b3 b4 b5 b6 b7.</code>	Introduces universally quantified variables or hypotheses into the proof context.
688	<code>rewrite keccak_lane_to_byte_bitsE b0 b1 b2 b3 b4 b5 b6 b7 /=. .</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
689	<code>rewrite keccak_pow2_1E keccak_pow2_2E keccak_pow2_6E.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
690	<code>by smt().</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
691	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
692	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
693	<code>lemma keccak_f1600_bytes_size st :</code>	States checked lemma <code>keccak_f1600_bytes_size</code> . The following proof script must derive this proposition from prior definitions and lemmas.
694	<code>size (keccak_f1600_bytes st) = keccak_state_bytes.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
695	<code>proof. by rewrite /keccak_f1600_bytes keccak_bytes_of_lanes_size. qed.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
696	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
697	<code>end HAETAE_Keccak1600.</code>	Closes the current theory or module block.